

LAPORAN TUGAS SISTEM OPERASI (PRAKTIKUM)
TUGAS P3.2: PERINTAH DASAR LINUX
CHAPTER 7: FILE PROPERTIES

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Sistem Operasi



Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	4
1.1 Lingkungan Pengerjaan	4
1.2 Cara Pengerjaan	4
1.3 Perbedaan Hasil dengan Buku	5
BAB II JAWABAN EXERCISE CHAPTER 7	6
2.1 Soal 1: Jumlah baris ke variabel	6
2.2 Soal 2: Menebak hasil wc -w	6
2.3 Soal 3: Baris dan byte dari berkas dan stdin	7
2.4 Soal 4: Daftar berkas dipisah null byte	7
2.5 Soal 5: Beda wc -c dan wc -m	8
2.6 Soal 6: Ukuran berkas .log dengan find+du dan ls	8
2.7 Soal 7: Ukuran isi folder dalam pangkat 1000	9
2.8 Soal 8: Opsi du --apparent-size	9
2.9 Soal 9: Kapan memakai df	10
2.10 Soal 10: Ukuran dua berkas dengan stat	10
2.11 Soal 11: touch tanpa membuat berkas baru	11
2.12 Soal 12: Waktu dari touch -t	11
2.13 Soal 13: Menyalin waktu dengan touch -r	12
2.14 Soal 14: Keluaran file tanpa nama berkas	12
2.15 Soal 15: basename dengan garis miring berlebih	13
2.16 Soal 16: Naik dua tingkat dengan dirname	13
2.17 Soal 17: Arti tiap karakter izin	14
2.18 Soal 18: Menebak hasil chmod 546	14
2.19 Soal 19: Izin direktori langsung dari mkdir	15
2.20 Soal 20: Mengubah izin jadi 220 dengan huruf	15
2.21 Soal 21: Mengubah izin direktori jadi 757 dengan huruf	16
BAB III PENUTUP	17

3.1 Catatan Hasil	17
3.2 Kesimpulan	17

BAB I PENDAHULUAN

Tugas P3.2 memakai bahan chapter 7 buku *Computing from the Command Line* karya Sundeep Agarwal, yaitu File Properties. Chapter ini membahas perintah untuk melihat dan mengubah sifat berkas: `wc` untuk menghitung baris, kata, dan byte, `du` dan `df` untuk ukuran berkas dan ruang disk, `stat` dan `touch` untuk informasi dan waktu berkas, `file` untuk jenis isi berkas, `basename` dan `dirname` untuk mengolah path, serta `chmod` untuk izin akses. Laporan ini memuat jawaban 21 soal exercise chapter 7 beserta tangkapan layar hasil percobaannya.

1.1 Lingkungan Pengerjaan

Seperti tugas-tugas sebelumnya, semua percobaan dijalankan di Ubuntu 24.04 yang berjalan di OrbStack, bukan di macOS, karena buku memakai GNU coreutils. Beberapa pengaturan mesin ikut dicatat karena berpengaruh langsung ke hasil soal chapter ini.

Tabel 1.1: Versi perangkat lunak dan pengaturan mesin

Komponen	Nilai
Sistem operasi	Ubuntu 24.04.5 LTS
Shell	GNU bash 5.2.21
<code>wc</code> , <code>du</code> , <code>df</code> , <code>stat</code> , <code>touch</code> , <code>chmod</code>	GNU coreutils 9.4
<code>find</code>	GNU findutils 4.9.0
<code>file</code>	file 5.45
Filesystem folder home	btrfs
Zona waktu	WIB (+0700)
<code>umask</code>	0022
Locale	C.UTF-8

Image Ubuntu bawaan OrbStack belum punya perintah `file`, jadi paketnya dipasang dulu untuk soal 14.

1.2 Cara Pengerjaan

1. Soal yang memakai berkas contoh dikerjakan di
`~/cli-computing/example_files/text_files/`.
2. Soal 6, 7, 8, dan 13 dikerjakan di folder `scripts` buku dengan `source du.sh` atau `source touch.sh`, sesuai petunjuk soalnya. Folder contohnya dibuat baru di soal 6 dan 13.
3. Soal yang membuat berkas atau folder baru (11, 12, 18 sampai 21) dikerjakan di folder latihan kosong `~/tugas3-so/ch7-NN`, sesuai saran buku untuk memakai direktori sementara.
4. Tangkapan layar diambil dari jendela Terminal yang benar-benar menjalankan perintahnya. Judul jendela memperlihatkan `orb -m ubuntu` dan prompt memperlihatkan `ghaisan@ubuntu` beserta folder kerjanya.

1.3 Perbedaan Hasil dengan Buku

Soal yang punya contoh keluaran di buku dan hasilnya sama persis adalah soal 1, 3, 4, 6, 10, 16, dan 19. Soal yang hasilnya berbeda, penyebabnya ada di lingkungan mesin, bukan di perintahnya:

1. Soal 7: ukuran direktori lebih kecil karena btrfs tidak menghitung blok untuk direktori.
2. Soal 12 dan 13: zona waktu tercetak +0700 (WIB), sedangkan buku +0530.
3. Soal 14: susunan kata keluaran `file` berbeda karena versinya berbeda.
4. Soal 17, 18, 20, dan 21: izin awal berkas baru 644 dan direktori baru 755, sedangkan di buku 664 dan 775. Penyebabnya umask mesin ini 0022, sedangkan buku 0002. Hasil akhir soal 18, 20, dan 21 tetap sama dengan buku.

BAB II JAWABAN EXERCISE CHAPTER 7

Beberapa soal hanya berupa pertanyaan atau meminta menebak keluaran. Untuk soal seperti itu jawabannya tetap dicoba di terminal supaya kelihatan bahwa jawabannya benar.

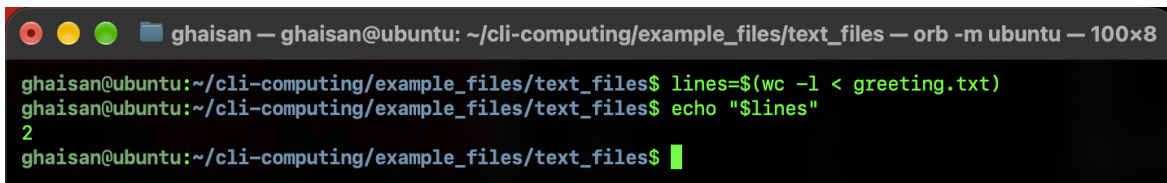
2.1 Soal 1: Jumlah baris ke variabel

Soal. Simpan jumlah baris berkas `greeting.txt` ke variabel `lines`.

Jawaban.

```
lines=$(wc -l < greeting.txt)
echo "$lines"
```

`wc -l` menghitung baris dan `$ ( )` menyimpan keluarannya ke variabel. Berkasnya sengaja diberikan lewat `<`. Kalau ditulis `wc -l greeting.txt`, keluarannya `2 greeting.txt` dan nama berkas ikut masuk ke variabel. Lewat `<`, `wc` membaca stdin dan tidak tahu nama berkasnya, jadi yang tercetak angkanya saja.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ lines=$(wc -l < greeting.txt)
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo "$lines"
2
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.1: hasil percobaan soal 1

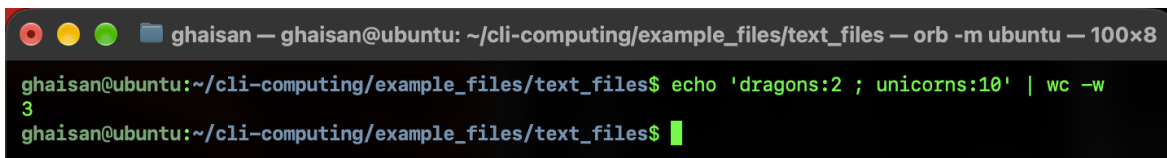
2.2 Soal 2: Menebak hasil `wc -w`

Soal. Apa keluaran `echo 'dragons:2 ; unicorns:10' | wc -w`?

Jawaban.

```
echo 'dragons:2 ; unicorns:10' | wc -w
```

Tebakan saya 3, dan hasil percobaan memang 3. `wc -w` menganggap kata sebagai deretan karakter yang dipisah spasi, tab, atau baris baru. Titik dua bukan pemisah, jadi `dragons:2`, `;`, dan `unicorns:10` masing-masing satu kata.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo 'dragons:2 ; unicorns:10' | wc -w
3
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.2: hasil percobaan soal 2

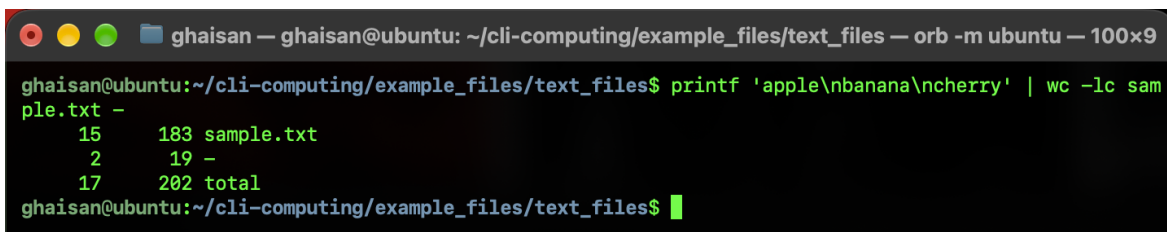
2.3 Soal 3: Baris dan byte dari berkas dan stdin

Soal. Pakai opsi dan argumen yang tepat supaya keluarannya berisi jumlah baris dan byte untuk `sample.txt`, `stdin`, dan totalnya.

Jawaban.

```
printf 'apple\nbanana\ncherry' | wc -lc sample.txt -
```

`-l` menghitung baris dan `-c` menghitung byte. Argumen `-` mewakili `stdin` dan ditulis sesudah `sample.txt` supaya urutannya sama dengan contoh. `Stdin` hanya terhitung 2 baris walaupun isinya tiga kata, karena `wc -l` sebenarnya menghitung karakter baris baru dan `printf` tidak menambahkan `\n` setelah `cherry`.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'apple\nbanana\ncherry' | wc -lc sample.txt -
15 183 sample.txt
2 19 -
17 202 total
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.3: hasil percobaan soal 3

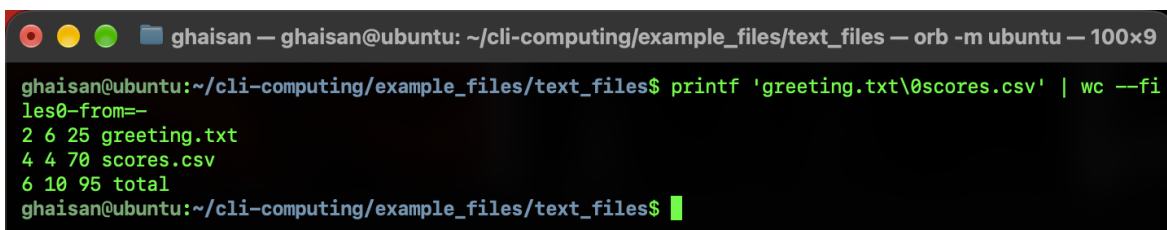
2.4 Soal 4: Daftar berkas dipisah null byte

Soal. Dari manual `wc`, cari cara supaya daftar nama berkas yang dipisah karakter null dari `stdin` bisa dihitung.

Jawaban.

```
printf 'greeting.txt\0scores.csv' | wc --files0-from=-
```

Opsi `--files0-from=-` membaca daftar nama berkas yang dipisah null byte, dan `-` berarti daftar itu diambil dari `stdin`. Tanpa opsi lain, `wc` mencetak tiga kolom: baris, kata, dan byte. Cara ini berguna kalau daftar berkasnya berasal dari `find -print0`.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'greeting.txt\0scores.csv' | wc --files0-from=-
2 6 25 greeting.txt
4 4 70 scores.csv
6 10 95 total
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.4: hasil percobaan soal 4

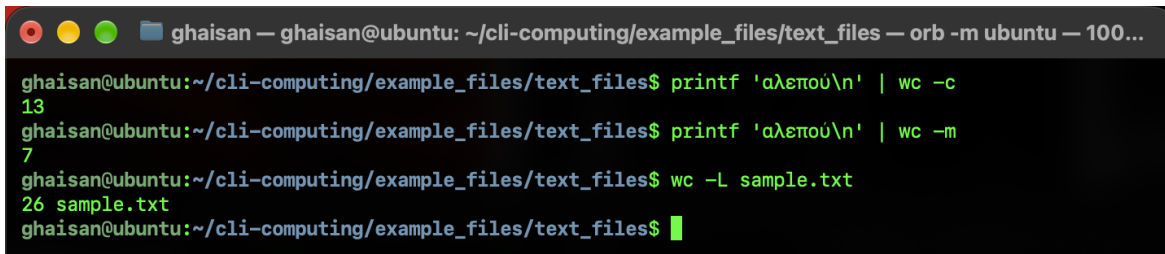
2.5 Soal 5: Beda `wc -c` dan `wc -m`

Soal. Apa beda opsi `wc -c` dan `wc -m`, dan opsi apa untuk mendapatkan panjang baris terpanjang?

Jawaban.

```
printf 'αλεπού\n' | wc -c
printf 'αλεπού\n' | wc -m
wc -L sample.txt
```

`-c` menghitung byte, sedangkan `-m` menghitung karakter. Bedanya baru terasa pada teks multibyte. Kata αλεπού berisi 6 huruf Yunani yang masing-masing 2 byte di UTF-8, ditambah satu baris baru, jadi hasilnya 13 byte tetapi 7 karakter. Untuk panjang baris terpanjang dipakai `-L`, dan di `sample.txt` hasilnya 26.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'αλεπού\n' | wc -c
13
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'αλεπού\n' | wc -m
7
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ wc -L sample.txt
26 sample.txt
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.5: hasil percobaan soal 5

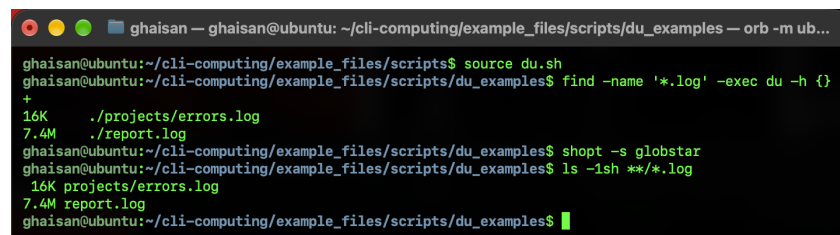
2.6 Soal 6: Ukuran berkas `.log` dengan `find+du` dan `ls`

Soal. Cari berkas berakhiran `.log` dan tampilkan ukurannya dalam format mudah dibaca, sekali dengan `find` dan `du`, sekali dengan `ls` dan fitur shell.

Jawaban.

```
source du.sh
find -name '*.log' -exec du -h {} +
shopt -s globstar
ls -lsh **/*.log
```

Pada cara pertama, `-exec du -h {} +` menjalankan `du` satu kali untuk semua berkas `.log` yang ditemukan, dan `-h` membuat ukurannya berbentuk K atau M. Pada cara kedua, `**` dari `globstar` membuat berkas di sub-folder ikut, `-s` menampilkan ukuran berkas, `-h` membuatnya mudah dibaca, dan `-l` membuat satu berkas satu baris. Hasil keduanya sama persis dengan buku.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ source du.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ find -name '*.log' -exec du -h {} +
+
16K  ./projects/errors.log
7.4M ./report.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ shopt -s globstar
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ ls -lsh **/*.log
16K projects/errors.log
7.4M report.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$
```

Gambar 2.6: hasil percobaan soal 6

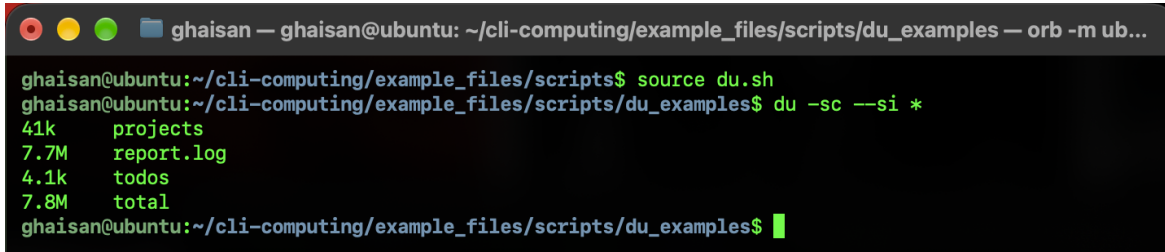
2.7 Soal 7: Ukuran isi folder dalam pangkat 1000

Soal. Tampilkan ukuran berkas dan folder di path aktif dalam pangkat 1000 tanpa rincian sub-folder, beserta totalnya.

Jawaban.

```
source du.sh
du -sc --si *
```

`-s` hanya mencetak ringkasan untuk tiap argumen, `-c` menambahkan baris total, dan `--si` memakai pangkat 1000 (k, M), bukan 1024 seperti `-h`. Angka `projects` dan `todos` lebih kecil dari buku (41k dan 4.1k, di buku 50k dan 8.2k). Selisihnya pas dengan satu blok 4 KiB untuk tiap direktori. Di ext4 setiap direktori memakan minimal satu blok, sedangkan di btrfs isi direktori disimpan di metadata filesystem sehingga `du` tidak menghitungnya sebagai blok milik direktori itu. Ukuran berkasnya sendiri sama.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ source du.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ du -sc --si *
41k    projects
7.7M   report.log
4.1k   todos
7.8M   total
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$
```

Gambar 2.7: hasil percobaan soal 7

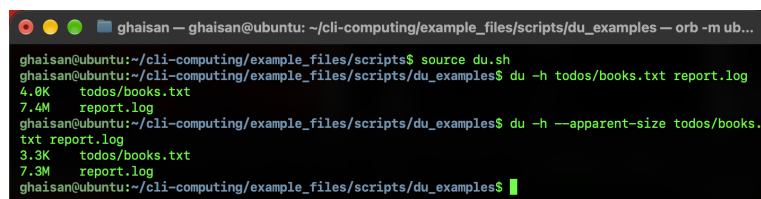
2.8 Soal 8: Opsi du --apparent-size

Soal. Apa fungsi opsi `du --apparent-size`?

Jawaban.

```
source du.sh
du -h todos/books.txt report.log
du -h --apparent-size todos/books.txt report.log
```

Secara bawaan `du` melaporkan ruang disk yang dipakai, yaitu jumlah blok yang dialokasikan. `--apparent-size` melaporkan ukuran isi berkas yang sebenarnya. Disk dialokasikan per blok 4 KiB, jadi `books.txt` yang isinya 3333 byte tetap memakan 4.0K, padahal ukuran nyatanya 3.3K. Kebalikannya bisa terjadi pada sparse file, yang ukuran nyatanya justru lebih besar dari ruang disk yang dipakai.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ source du.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ du -h todos/books.txt report.log
4.0K   todos/books.txt
7.4M   report.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$ du -h --apparent-size todos/books.txt report.log
3.3K   todos/books.txt
7.3M   report.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/du_examples$
```

Gambar 2.8: hasil percobaan soal 8

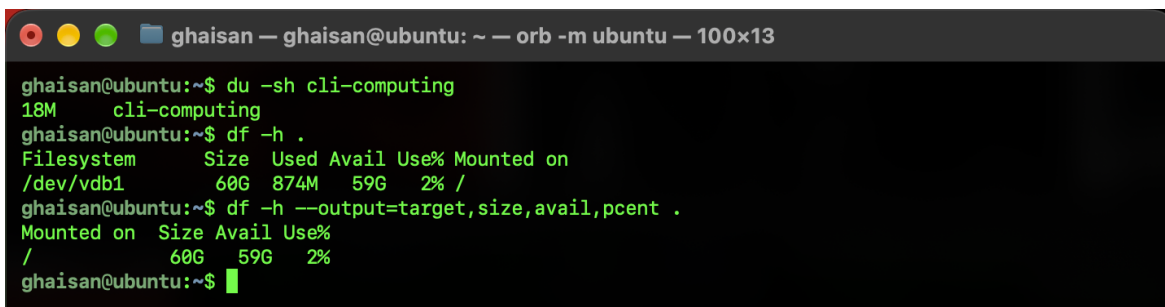
2.9 Soal 9: Kapan memakai df

Soal. Kapan `df` dipakai daripada `du`, dan opsi `df` apa untuk menampilkan kolom tertentu saja?

Jawaban.

```
du -sh cli-computing
df -h .
df -h --output=target,size,avail,pcent .
```

`du` menjumlahkan ukuran berkas dan folder tertentu dengan cara menelusurinya. `df` melaporkan kondisi filesystem secara keseluruhan: kapasitas, yang terpakai, dan sisa ruang. Jadi `df` dipakai kalau pertanyaannya apakah disk masih cukup, sedangkan `du` kalau ingin tahu folder mana yang paling besar. Contoh di atas memperlihatkan bedanya: `du` hanya menghitung folder `cli-computing` (18M), sedangkan `df` menampilkan seluruh partisi root 60G. Kolom yang ingin ditampilkan dipilih dengan `--output`.



```
ghaisan@ubuntu:~$ du -sh cli-computing
18M    cli-computing
ghaisan@ubuntu:~$ df -h .
Filesystem      Size  Used Avail Use% Mounted on
/dev/vdb1        60G   874M   59G   2% /
ghaisan@ubuntu:~$ df -h --output=target,size,avail,pcent .
Mounted on  Size Avail Use%
/           60G   59G   2%
```

Gambar 2.9: hasil percobaan soal 9

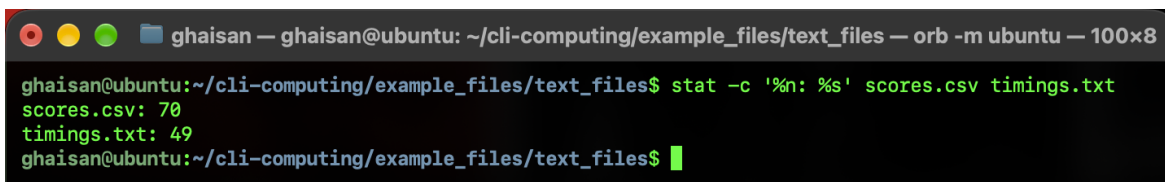
2.10 Soal 10: Ukuran dua berkas dengan stat

Soal. Tampilkan ukuran `scores.csv` dan `timings.txt` dengan format nama: ukuran.

Jawaban.

```
stat -c '%n: %s' scores.csv timings.txt
```

`stat -c` menerima format sendiri. `%n` diganti nama berkas dan `%s` diganti ukurannya dalam byte, sedangkan titik dua dan spasi di antaranya dicetak apa adanya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ stat -c '%n: %s' scores.csv timings.txt
scores.csv: 70
timings.txt: 49
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.10: hasil percobaan soal 10

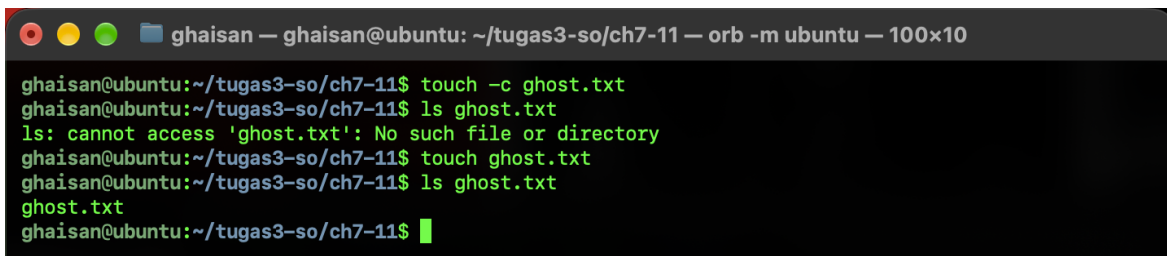
2.11 Soal 11: touch tanpa membuat berkas baru

Soal. Opsi touch apa yang mencegah berkas dibuat kalau berkasnya belum ada?

Jawaban.

```
touch -c ghost.txt
ls ghost.txt
touch ghost.txt
ls ghost.txt
```

Opsinya `-c` atau `--no-create`. Terlihat `ls` gagal menemukan `ghost.txt` setelah `touch -c`, karena berkasnya memang tidak dibuat. Tanpa `-c`, `touch` langsung membuat berkas kosong.

A terminal window titled 'ghaisan — ghaisan@ubuntu: ~/tugas3-so/ch7-11 — orb -m ubuntu — 100x10'. The terminal shows the following commands and output:

```
ghaisan@ubuntu:~/tugas3-so/ch7-11$ touch -c ghost.txt
ghaisan@ubuntu:~/tugas3-so/ch7-11$ ls ghost.txt
ls: cannot access 'ghost.txt': No such file or directory
ghaisan@ubuntu:~/tugas3-so/ch7-11$ touch ghost.txt
ghaisan@ubuntu:~/tugas3-so/ch7-11$ ls ghost.txt
ghost.txt
ghaisan@ubuntu:~/tugas3-so/ch7-11$
```

Gambar 2.11: hasil percobaan soal 11

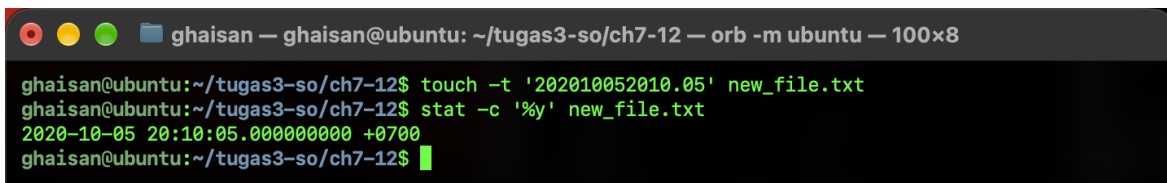
2.12 Soal 12: Waktu dari touch -t

Soal. Kalau `new_file.txt` belum ada, apa keluaran `stat -c '%y'` setelah `touch -t '202010052010.05'`?

Jawaban.

```
touch -t '202010052010.05' new_file.txt
stat -c '%y' new_file.txt
```

Format `-t` adalah `[[CC]YY]MMDDhhmm[.ss]`, jadi `202010052010.05` berarti 5 Oktober 2020 pukul 20.10 lewat 5 detik. Berkasnya belum ada sehingga dibuat dengan waktu itu. Bagian nanodetik bernilai nol karena `-t` hanya sampai detik, dan zona waktunya `+0700` karena mesin ini memakai WIB.

A terminal window titled 'ghaisan — ghaisan@ubuntu: ~/tugas3-so/ch7-12 — orb -m ubuntu — 100x8'. The terminal shows the following commands and output:

```
ghaisan@ubuntu:~/tugas3-so/ch7-12$ touch -t '202010052010.05' new_file.txt
ghaisan@ubuntu:~/tugas3-so/ch7-12$ stat -c '%y' new_file.txt
2020-10-05 20:10:05.000000000 +0700
ghaisan@ubuntu:~/tugas3-so/ch7-12$
```

Gambar 2.12: hasil percobaan soal 12

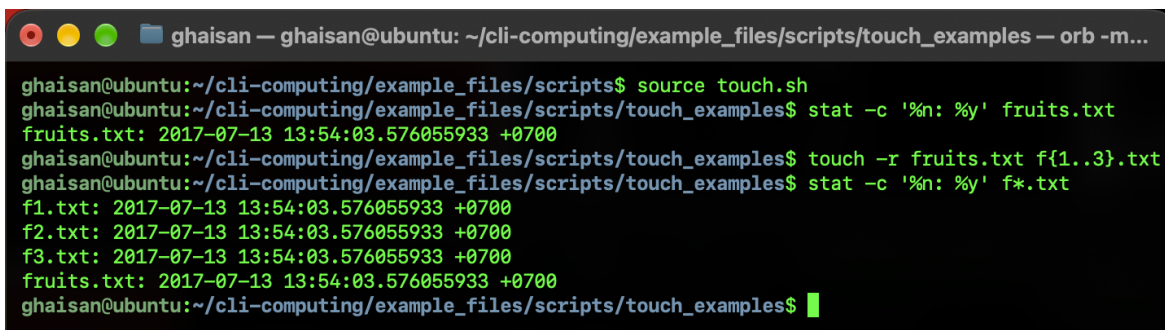
2.13 Soal 13: Menyalin waktu dengan touch -r

Soal. Apakah `touch -r fruits.txt f{1..3}.txt` valid, dan apa keluaran `stat` sesudahnya?

Jawaban.

```
source touch.sh
stat -c '%n: %y' fruits.txt
touch -r fruits.txt f{1..3}.txt
stat -c '%n: %y' f*.txt
```

Valid. `-r` (reference) menyalin waktu akses dan waktu modifikasi dari `fruits.txt` ke berkas lain, dan brace expansion menghasilkan `f1.txt` sampai `f3.txt`. Karena ketiganya belum ada, `touch` membuatnya dengan waktu yang sama persis sampai nanodetik. Pola `f*.txt` juga cocok dengan `fruits.txt`, jadi keluarannya empat baris. Zona waktunya `+0700`, bukan `+0530` seperti di buku, karena `touch.sh` menulis tanggal tanpa zona waktu sehingga dibaca sebagai waktu lokal mesin.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/touch_examples$ source touch.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/touch_examples$ stat -c '%n: %y' fruits.txt
fruits.txt: 2017-07-13 13:54:03.576055933 +0700
ghaisan@ubuntu:~/cli-computing/example_files/scripts/touch_examples$ touch -r fruits.txt f{1..3}.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/touch_examples$ stat -c '%n: %y' f*.txt
f1.txt: 2017-07-13 13:54:03.576055933 +0700
f2.txt: 2017-07-13 13:54:03.576055933 +0700
f3.txt: 2017-07-13 13:54:03.576055933 +0700
fruits.txt: 2017-07-13 13:54:03.576055933 +0700
ghaisan@ubuntu:~/cli-computing/example_files/scripts/touch_examples$
```

Gambar 2.13: hasil percobaan soal 13

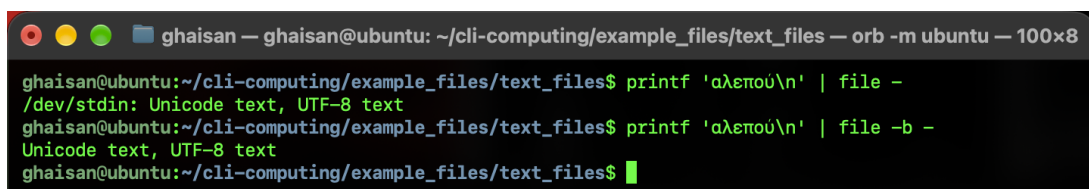
2.14 Soal 14: Keluaran file tanpa nama berkas

Soal. Pakai opsi yang tepat supaya `file` tidak mencetak nama berkas di depan hasilnya.

Jawaban.

```
printf 'αλεπού\n' | file -
printf 'αλεπού\n' | file -b -
```

`-b` (brief) menghilangkan awalan nama berkas, dan `-` berarti yang diperiksa adalah stdin. Teks hasilnya Unicode text, UTF-8 text, sedangkan di buku UTF-8 Unicode text. Isinya sama, hanya susunan kata di `file 5.45` yang dipakai di sini berbeda dengan versi di buku.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'αλεπού\n' | file -
/dev/stdin: Unicode text, UTF-8 text
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'αλεπού\n' | file -b -
Unicode text, UTF-8 text
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.14: hasil percobaan soal 14

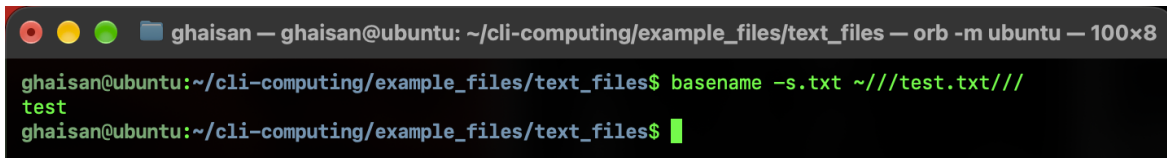
2.15 Soal 15: basename dengan garis miring berlebih

Soal. Apakah `basename -s.txt ~///test.txt///` valid, dan apa keluarannya?

Jawaban.

```
basename -s.txt ~///test.txt///
```

Valid, keluarannya `test`. Shell melebarkan `~` menjadi folder home dulu. `basename` lalu membuang garis miring di belakang, mengambil komponen terakhir yaitu `test.txt`, dan `-s.txt` membuang akhiran `.txt`. Garis miring yang berderet tidak jadi masalah.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ basename -s.txt ~///test.txt///
test
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.15: hasil percobaan soal 15

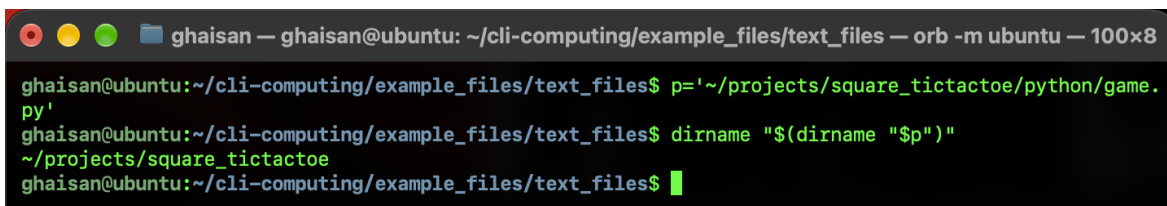
2.16 Soal 16: Naik dua tingkat dengan dirname

Soal. Dari path di variabel `p`, dapatkan `~/projects/square_tictactoe`.

Jawaban.

```
p='~/projects/square_tictactoe/python/game.py'
dirname "${dirname "$p"}"
```

`dirname` membuang komponen terakhir sebuah path, jadi untuk naik dua tingkat dipanggil dua kali. Yang dalam menghasilkan `~/projects/square_tictactoe/python`, lalu yang luar membuang `python`. Tanda `~` tidak dilebarkan karena saat mengisi `p` ditulis di dalam petik tunggal.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ p='~/projects/square_tictactoe/python/game.py'
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ dirname "${dirname "$p"}"
~/projects/square_tictactoe
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.16: hasil percobaan soal 16

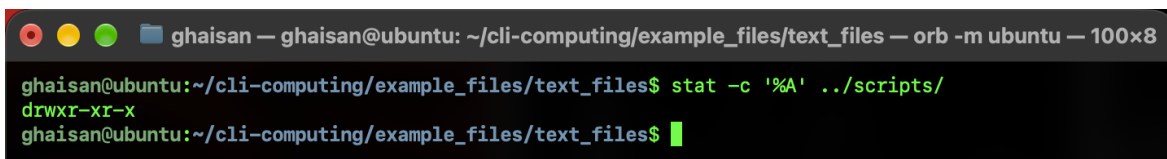
2.17 Soal 17: Arti tiap karakter izin

Soal. Jelaskan arti tiap karakter pada keluaran `stat -c '%A' ../scripts/`, di buku `drwxrwxr-x`.

Jawaban.

```
stat -c '%A' ../scripts/
```

Karakter pertama `d` adalah jenis berkas, yaitu direktori (berkas biasa ditulis `-`, symbolic link `l`). Sembilan karakter berikutnya dibagi tiga kelompok: pemilik, grup, dan pengguna lain. Tiap kelompok berisi `r` (baca), `w` (tulis), dan `x` (eksekusi), dan `-` berarti izin itu tidak ada. Pada direktori, `r` berarti boleh melihat daftar isinya, `w` boleh membuat, menghapus, atau mengganti nama isinya, dan `x` boleh masuk dengan `cd` dan mengakses isinya. Jadi `drwxrwxr-x` artinya pemilik dan grup punya akses penuh, sedangkan pengguna lain hanya boleh membaca dan masuk. Di mesin ini hasilnya `drwxr-xr-x` karena `umask 0022` mencabut izin tulis grup saat repo buku di-clone.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ stat -c '%A' ../scripts/
drwxr-xr-x
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.17: hasil percobaan soal 17

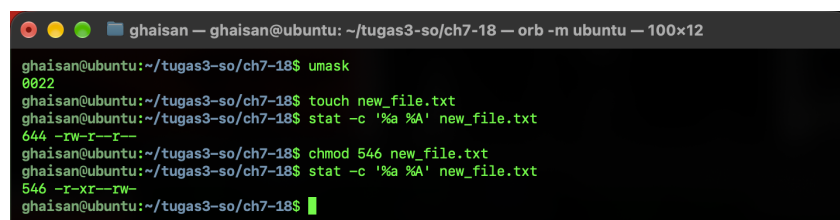
2.18 Soal 18: Menebak hasil `chmod 546 new_file.txt`

Soal. Apa keluaran `stat -c '%a %A' new_file.txt` setelah `chmod 546 new_file.txt`?

Jawaban.

```
umask
touch new_file.txt
stat -c '%a %A' new_file.txt
chmod 546 new_file.txt
stat -c '%a %A' new_file.txt
```

Jawabannya `546 -r-xr--rw-`. Tiap digit adalah jumlah dari `r = 4`, `w = 2`, dan `x = 1`. Digit `5 = 4 + 1` (`r-x`) untuk pemilik, `4` (`r--`) untuk grup, dan `6 = 4 + 2` (`rw-`) untuk pengguna lain. Mode angka menimpa semua izin sekaligus, jadi hasilnya tidak bergantung pada izin awal. Izin awalnya di sini `644`, bukan `664` seperti di buku, karena `umask` mesin ini `0022`.



```
ghaisan@ubuntu:~/tugas3-so/ch7-18$ umask
0022
ghaisan@ubuntu:~/tugas3-so/ch7-18$ touch new_file.txt
ghaisan@ubuntu:~/tugas3-so/ch7-18$ stat -c '%a %A' new_file.txt
644 -rw-r--r--
ghaisan@ubuntu:~/tugas3-so/ch7-18$ chmod 546 new_file.txt
ghaisan@ubuntu:~/tugas3-so/ch7-18$ stat -c '%a %A' new_file.txt
546 -r-xr--rw-
ghaisan@ubuntu:~/tugas3-so/ch7-18$
```

Gambar 2.18: hasil percobaan soal 18

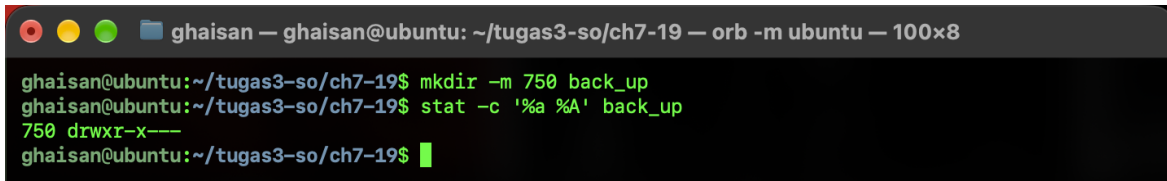
2.19 Soal 19: Izin direktori langsung dari mkdir

Soal. Buat direktori `back_up` dengan izin 750 memakai `mkdir` saja.

Jawaban.

```
mkdir -m 750 back_up
stat -c '%a %A' back_up
```

Opsi `-m` (`--mode`) memberi izin saat direktori dibuat, jadi tidak perlu `chmod` terpisah.



```
ghaisan — ghaisan@ubuntu: ~/tugas3-so/ch7-19 — orb -m ubuntu — 100x8
ghaisan@ubuntu:~/tugas3-so/ch7-19$ mkdir -m 750 back_up
ghaisan@ubuntu:~/tugas3-so/ch7-19$ stat -c '%a %A' back_up
750 drwxr-x---
```

Gambar 2.19: hasil percobaan soal 19

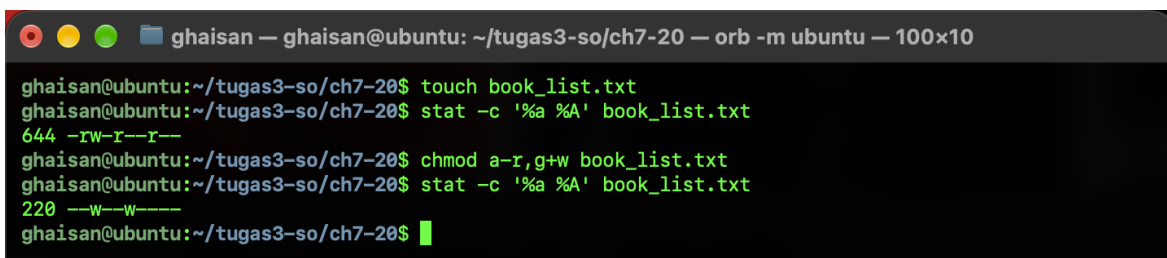
2.20 Soal 20: Mengubah izin jadi 220 dengan huruf

Soal. Ubah izin `book_list.txt` menjadi `--w--w----` tanpa menulis angka 220.

Jawaban.

```
touch book_list.txt
stat -c '%a %A' book_list.txt
chmod a-r,g+w book_list.txt
stat -c '%a %A' book_list.txt
```

`a-r` mencabut izin baca dari semua (pemilik, grup, lainnya), lalu `g+w` menambahkan izin tulis ke grup. Di mesin ini izin awalnya 644, jadi `a-r` menghasilkan 200 dan `g+w` menjadikannya 220. Kalau izin awalnya 664 seperti di buku, `a-r` langsung menghasilkan 220 dan `g+w` tidak mengubah apa-apa. Jadi perintah ini benar untuk kedua kondisi awal.



```
ghaisan — ghaisan@ubuntu: ~/tugas3-so/ch7-20 — orb -m ubuntu — 100x10
ghaisan@ubuntu:~/tugas3-so/ch7-20$ touch book_list.txt
ghaisan@ubuntu:~/tugas3-so/ch7-20$ stat -c '%a %A' book_list.txt
644 -rw-r--r--
ghaisan@ubuntu:~/tugas3-so/ch7-20$ chmod a-r,g+w book_list.txt
ghaisan@ubuntu:~/tugas3-so/ch7-20$ stat -c '%a %A' book_list.txt
220 --w--w----
```

Gambar 2.20: hasil percobaan soal 20

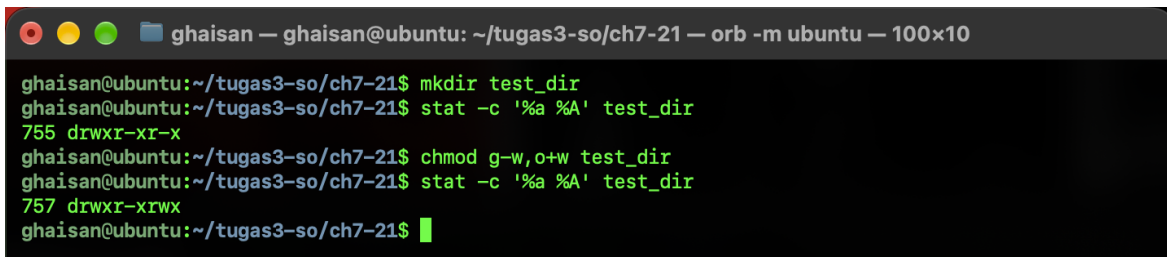
2.21 Soal 21: Mengubah izin direktori jadi 757 dengan huruf

Soal. Ubah izin `test_dir` menjadi `drwxr-xrwx` tanpa menulis angka 757.

Jawaban.

```
mkdir test_dir
stat -c '%a %A' test_dir
chmod g-w,o+w test_dir
stat -c '%a %A' test_dir
```

`g-w` mencabut izin tulis grup dan `o+w` menambahkan izin tulis untuk pengguna lain. Dari 775 di buku jadinya 755 lalu 757. Di mesin ini izin awalnya sudah 755, jadi `g-w` tidak mengubah apa-apa dan hanya `o+w` yang bekerja. Hasil akhirnya tetap 757 `drwxr-xrwx`.

A screenshot of a terminal window with a dark background. The window title is "ghaisan — ghaisan@ubuntu: ~/tugas3-so/ch7-21 — orb -m ubuntu — 100x10". The terminal shows the following commands and their outputs:
1. `ghaisan@ubuntu:~/tugas3-so/ch7-21$ mkdir test_dir` (no output)
2. `ghaisan@ubuntu:~/tugas3-so/ch7-21$ stat -c '%a %A' test_dir`
 Output: `755 drwxr-xr-x`
3. `ghaisan@ubuntu:~/tugas3-so/ch7-21$ chmod g-w,o+w test_dir` (no output)
4. `ghaisan@ubuntu:~/tugas3-so/ch7-21$ stat -c '%a %A' test_dir`
 Output: `757 drwxr-xrwx`
5. `ghaisan@ubuntu:~/tugas3-so/ch7-21$` (prompt with a green cursor)

Gambar 2.21: hasil percobaan soal 21

BAB III PENUTUP

3.1 Catatan Hasil

1. Hasil yang berbeda dengan buku semuanya bisa dijelaskan dari lingkungan mesin: filesystem, zona waktu, versi `file`, dan `umask`. Karena itu pengaturan mesin perlu dicatat sebelum membandingkan hasil.
2. `du` dan `ls -s` melaporkan ruang disk yang dipakai, bukan ukuran isi berkas. Ukuran isi didapat dari `du --apparent-size`, `stat -c '%s'`, atau `wc -c`.
3. `wc -l` menghitung karakter baris baru, jadi baris terakhir tanpa `\n` tidak ikut terhitung.
4. Mode huruf pada `chmod (a-r, g+w)` mengubah izin relatif terhadap izin awal, sedangkan mode angka menimpa semuanya. Mode huruf lebih aman kalau izin awalnya bisa berbeda antar mesin, seperti terlihat di soal 20 dan 21.

3.2 Kesimpulan

Chapter 7 memperlihatkan bahwa sebuah berkas punya banyak sifat selain isinya, yaitu ukuran, waktu akses dan modifikasi, jenis isi, letak di dalam path, dan izin akses. Tiap sifat punya perintahnya sendiri, dan `stat` bisa menampilkan hampir semuanya dengan format bebas. Soal `chmod` dan `umask` paling berguna untuk mata kuliah ini, karena izin akses berkas adalah salah satu cara sistem operasi melindungi berkas milik pengguna satu dari pengguna lain.